

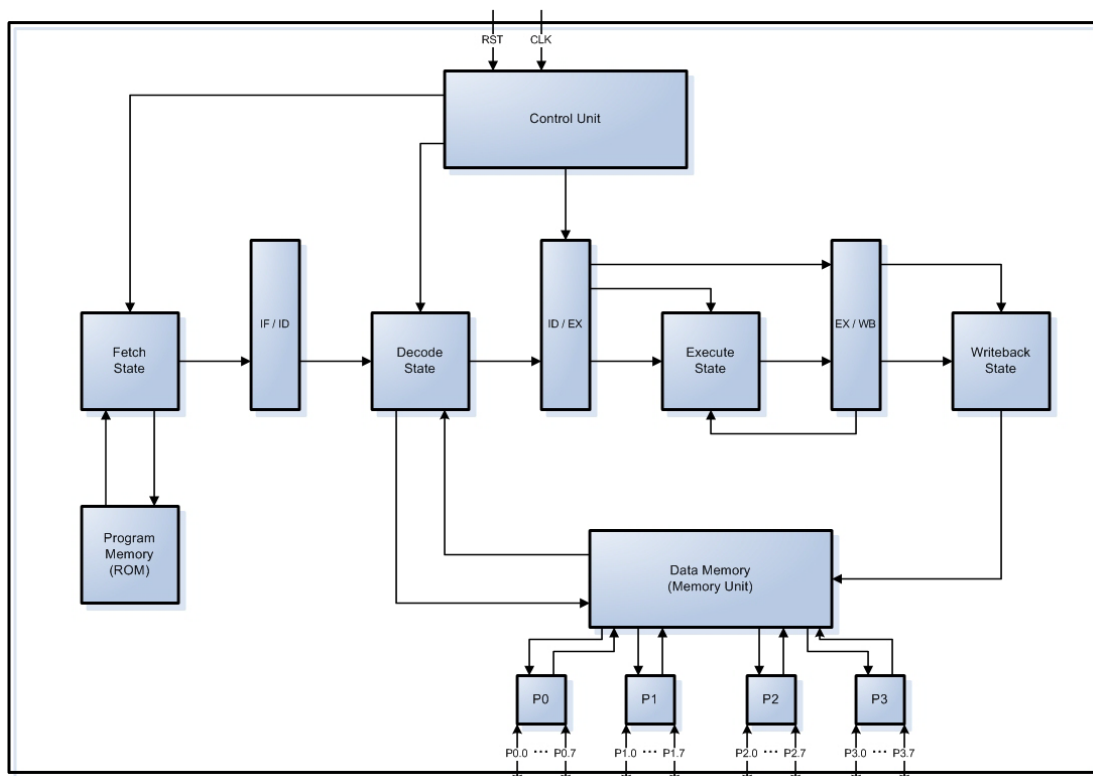
บทที่ 4

การออกแบบไปป์ไลน์

4.1 การออกแบบไมโครคอนโทรลเลอร์

ไมโครคอนโทรลเลอร์ที่ทำการออกแบบอ้างอิงการทำงานตามไมโครคอนโทรลเลอร์ต้นแบบเบอร์ 8051 ของอินเทลโดยมีโครงสร้างการทำงานและรายละเอียดการทำงานสามารถอ่านได้ตามภาคผนวก ก. ส่วนไมโครคอนโทรลเลอร์ที่ทำการออกแบบเป็นส่วนย่อยของไมโครคอนโทรลเลอร์ดังกล่าว มีโครงสร้างการทำงานดังรูปที่ 4.1 โดยแบ่งการทำงานออกเป็นบล็อก ตามสแตจการทำงานจำนวน 4 บล็อก แต่ละบล็อกจะทำงานแยกจากกันบนข้อมูลของคำสั่งแต่ละคำสั่งในลำดับการทำงานที่เรียงต่อเนื่องกันไป ซึ่งจะอธิบายการทำงานของแต่ละบล็อกโดยละเอียดในส่วนถัดไป และมี Storage Buffer กันแต่ละสแตจไว้ ส่วนอื่นที่สำคัญของไมโครคอนโทรลเลอร์ที่ทำการออกแบบ มีดังนี้

คอนโทรลยูนิต(Control Unit) ใช้สำหรับควบคุมการทำงานทั้งหมดของไมโครคอนโทรลเลอร์ โดยการส่งสัญญาณควบคุมไปยังส่วนของบล็อกอื่นๆ ที่อยู่ภายใน



รูปที่ 4.1 โครงสร้างของไมโครคอนโทรลเลอร์ที่ทำการออกแบบ

หน่วยความจำข้อมูล(Data Memory) เป็นหน่วยความจำที่ใช้เก็บข้อมูลในตำแหน่งต่างๆ ทั้งแบบไบต์และบิต และเป็นส่วนของรีจิสเตอร์ภายในต่างๆ ด้วย

หน่วยความจำโปรแกรม เป็นหน่วยความจำส่วนที่ใช้ในการเก็บโปรแกรมสำหรับการทำงานของไมโครคอนโทรลเลอร์ และส่วนของรีจิสเตอร์พอร์ตที่ใช้ควบคุมการทำงานของพอร์ตโดยมีทั้งสิ้น 4 พอร์ตพอร์ตละ 8 บิต

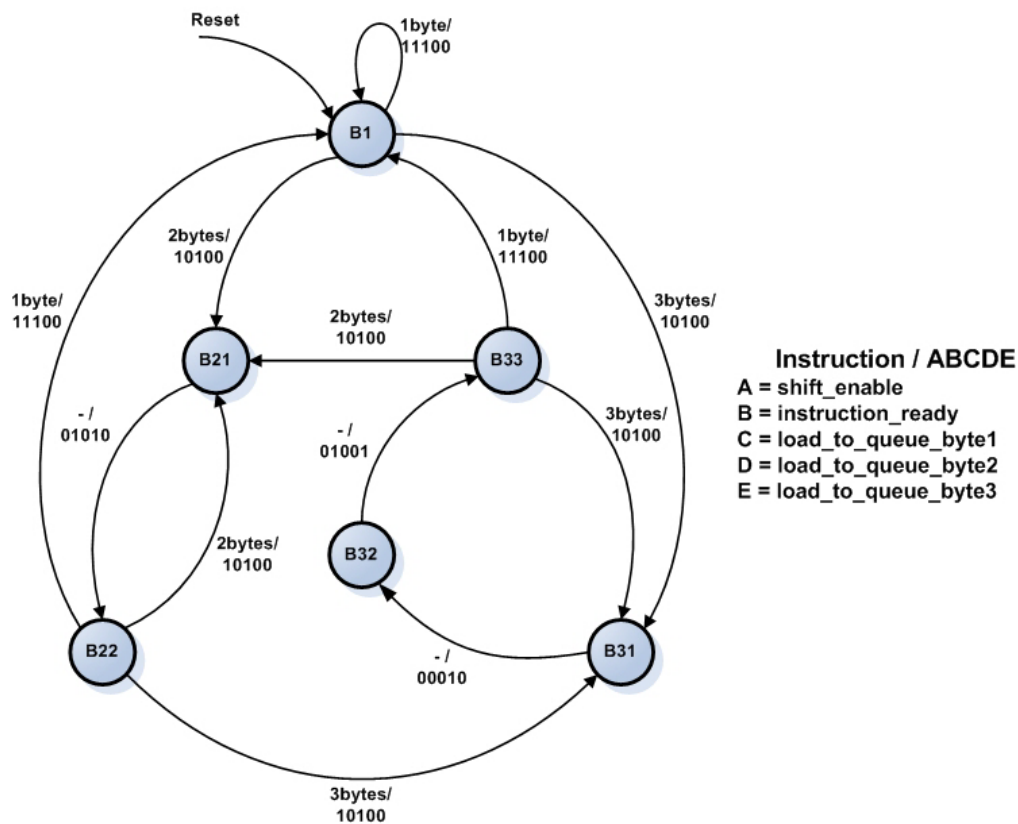
4.2 การแบ่งสแต็กการทำงาน

การแบ่งสแต็กการทำงานของไปป์ไลน์ให้มีจำนวนมากขึ้น สามารถช่วยเพิ่มประสิทธิภาพในการทำงานให้รวดเร็วขึ้นได้จริง แต่ยังคงมีการพิจารณาถึงปัจจัยอื่นๆประกอบในการกำหนดจำนวนของสแต็กในการทำงานของชิพอีกด้วย ทั้งการควบคุมข้อมูลที่ส่งต่อระหว่างสแต็กแต่ละสแต็ก และขนาดของวงจรควบคุม จากการศึกษาจากเอกสารและผลจากการทดลอง ในการแบ่งสแต็กการทำงานของไมโครคอนโทรลเลอร์ต้นแบบออกเป็น 5 สแต็กโดยมีสแต็กการถอดรหัสแยกกันสองสแต็กเพื่อทำการเข้าถึงหน่วยความจำสองครั้ง ในกรณีที่มีการทำคำสั่งแบบเข้าถึงข้อมูลทางอ้อมนั้น พบว่าสแต็กการถอดรหัสทั้งสองนั้นสามารถทำงานร่วมกันได้ โดยไม่ทำให้การทำงานช้าลง ดังนั้นจึงทำการแบ่งสแต็กการทำงานของไมโครคอนโทรลเลอร์ออกเป็น 4 สแต็ก ได้แก่ สแต็กการอ่านคำสั่ง สแต็กการถอดรหัสคำสั่ง สแต็กการทำคำสั่ง และสแต็กการเขียนผลลัพธ์ เพื่อเป็นการลดการควบคุมข้อมูลและความซับซ้อนของวงจรควบคุมการทำงานภายในของไมโครคอนโทรลเลอร์ที่ออกแบบ เนื่องจากการมีจำนวนสแต็กไม่มาก รายละเอียดในการทำงานของแต่ละสแต็กมีดังนี้

4.2.1 สแต็กการถอดรหัสคำสั่ง (Fetch Stage)

ทำหน้าที่ในการอ่านคำสั่งจากหน่วยความจำโปรแกรมส่งให้กับไปป์ไลน์ โดยอาศัยการทำงานของตัวจัดลำดับคำสั่ง (Instruction Queue) อ่านและจัดลำดับคำสั่งที่สมบูรณ์ส่งไปยังส่วนถัดไปของไปป์ไลน์ การทำงานของตัวจัดลำดับคำสั่งจะอยู่ในรูปแบบของไฟน์ดัสแต็กเมชชีน อ่านคำสั่งในการทำงานของไมโครคอนโทรลเลอร์ต้นแบบซึ่ง โดยมีคำสั่งที่แตกต่างกันอยู่ 3 รูปแบบตามความยาวของคำสั่ง ตั้งแต่ 1 – 3 ไบต์เมื่อครบแล้วจึงส่งต่อไปเก็บไว้ในส่วนของรีจิสเตอร์เก็บข้อมูลคำสั่งเพื่อส่งไปยังสแต็กถัดไป หากคำสั่งที่อ่านมายังไม่ครบตามจำนวน ตัวจัดลำดับคำสั่งจะทำการป้อนคำสั่ง NOP (No Operation) ให้กับไปป์ไลน์เพื่อให้ไปป์ไลน์มีคำสั่งในการทำงานครบอยู่ตลอดเวลา ไม่เกิดความผิดพลาดขึ้นในไปป์ไลน์ สแต็กเมชชีนแสดงการทำงานของตัวจัดลำดับคำสั่งแสดงดังรูปที่ 4.2

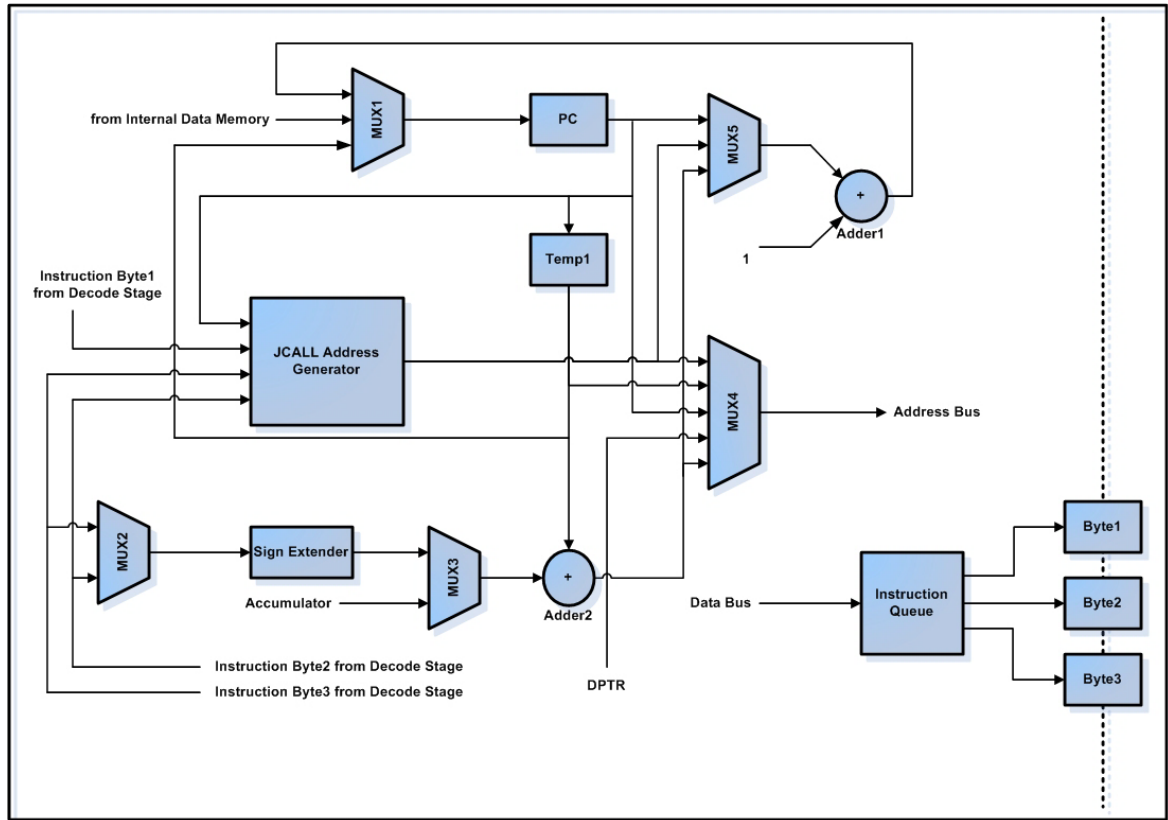
ภายในสแต็กการอ่านคำสั่งจะมีรีจิสเตอร์พีซี (Program Counter) สำหรับเก็บค่าตำแหน่งของหน่วยความจำโปรแกรมที่ทำงานอยู่ในปัจจุบัน โดยใช้ MUX1 ในการกำหนดข้อมูลที่จะเข้าสู่พีซีในการทำคำสั่งถัดไป โดยมีกรณีที่เกิดขึ้นในการอัปเดตค่าของพีซีได้ 3 กรณี คือ กรณีแรกเป็นการอัปเดตค่าในการทำงานเรียงตามลำดับปกติของโปรแกรม กรณีที่สองเป็นการรับค่ามาจากหน่วยความจำข้อมูลภายในเมื่อเกิดการทำคำสั่ง RET หรือ RETI ซึ่งต้องโหลดค่ากลับมาจากสแต็ก กรณีสุดท้ายคือการโหลดค่าของพีซีเก่าเพื่อย้อนไปอ่านคำสั่งที่ผ่านมาเมื่อพบว่าคำสั่ง



รูปที่ 4.2 Finite State Machine ของ Instruction Decoder

ก่อนหน้านี้เป็นคำสั่งกระโดดแล้ว จะสั่งให้มีการหยุดการอ่านคำสั่งและทิ้งคำสั่งที่อ่านมาก่อนหน้านี้ เนื่องจากมีการอ่านคำสั่งออกไปยังตัวจัดลำดับคำสั่งแล้วและค่าพีซีมีการเปลี่ยนไปแล้วด้วย จึงต้องทำการดึงค่าเดิมกลับมาอีกครั้ง จากระจิสเตอร์ Temp1 เพื่อให้สามารถอ่านคำสั่งเดิมได้ใหม่หลังจากการทำคำสั่งกระโดดเสร็จแล้ว แสดงการอ่านคำสั่งยังมีส่วนที่ทำงานเกี่ยวกับการจัดการกลุ่มคำสั่งกระโดด โดยอาศัยการทำงานของ JCALL Address Generator ในการหาค่าตำแหน่งแอดเดรสของคำสั่งที่ทำการกระโดดโดยนำคำสั่งไบต์ที่ 1 และ 2 มาทำการหาค่าตำแหน่งปลายทางของการกระโดดในกรณีที่คำสั่งเป็นคำสั่ง ACALL และ AJMP และนำคำสั่งไบต์ที่ 2 และ 3 ของคำสั่ง LCALL และ LJMP มาหาค่าตำแหน่งปลายทางของการกระโดดซึ่งเป็นการกระโดดแบบไม่มีเงื่อนไข เมื่อมีการทำคำสั่งกระโดดแบบมีเงื่อนไข Adder2 ถูกใช้สำหรับการคำนวณตำแหน่งปลายทางของการกระโดดแบบมีเงื่อนไข เมื่อมีการกำหนดค่าของตำแหน่งปลายทางไว้ที่คำสั่งไบต์ที่ 2 หรือ 3 และเนื่องจากค่าของตำแหน่งดังกล่าวมีขนาด 8 บิต จึงต้องแปลงด้วยการใช้ Sign Extender เป็น 16 บิตก่อนบวกกับค่าพีซี นอกจากนี้ยังใช้ Adder2 ในการทำคำสั่ง MOVC A,@A+PC ซึ่งต้องทำการหาค่าของตำแหน่งที่ต้องการจากการบวกค่าในรีจิสเตอร์แอดคิวมูเลเตอร์กับค่า

ของพีซี และต้องมีการเก็บค่าพีซีนี้ไว้ในรีจิสเตอร์ Temp1 ก่อนด้วยเช่นเดียวกับ ข้อมูลสำคัญของ สเตจการอ่าน คำสั่งคือคำสั่ง ไบต์ที่ 1, 2 และ 3 เส้นทางหลักของข้อมูลของสเตจการอ่านคำสั่ง แสดงดังรูปที่ 4.3



รูปที่ 4.3 เส้นทางหลักของข้อมูลในสเตจการอ่านคำสั่ง

4.2.2 สเตจการถอดรหัสคำสั่ง (Decode Stage)

สเตจการถอดรหัสคำสั่งได้รับอินพุตเป็นคำสั่งที่สมบูรณ์ที่มีขนาดตั้งแต่ 1 ถึง 3 ไบต์ซึ่งรองรับการทำงานในแอดเดรสซึ่งโหมดที่แตกต่างกัน 6 โหมดได้แก่ Immediate, Direct, Indirect, Indexed, Register และ Bit การทำงานของสเตจนี้จึงเป็นการถอดรหัสคำสั่งเพื่อเลือกโหมด และทำการเข้าถึงข้อมูลภายในหน่วย ความจำหรือรีจิสเตอร์ใช้งานทั่วไป นอกจากนี้ยังทำการถอดคำสั่งไบต์ที่ 1 ซึ่งเป็นตัวกำหนดการทำงานของหน่วยประมวลผล (ALU) ในสเตจถัดไป โดยส่งไปเก็บไว้ยังรีจิสเตอร์ ALU_operation นอกจากนี้สเตจการถอดรหัสคำสั่งยังรับผิดชอบการหาตำแหน่งของต้นทางและปลายทางของคำสั่งที่จะทำต่อไปด้วย

คำสั่งส่วนใหญ่เป็นคำสั่งที่มีตำแหน่งปลายทางที่เดียวเท่านั้น ยกเว้นคำสั่งบางคำสั่งที่มีตำแหน่งปลายทางสองแห่งเช่น XCH, XCHD, MUL, DIV ค่าของต้นทางและปลายทางของโอเปอร์เรนด์แต่ละตัวจะถูกเก็บอยู่ในรีจิสเตอร์ S1, S2, D1 และ D2 ตามลำดับ ซึ่งใช้ในการส่งค่าผลของการทำคำสั่งในการทำฟอร์เวิร์ดโอเปอร์เรนด์เมื่อเกิดปัญหา RAW ในการทำงานของไปป์ไลน์ และค่าของ D1 และ D2 ยังใช้ในการกำหนดตำแหน่งปลายทางใน

การเขียนข้อมูลของสแต็กการเขียนผลลัพธ์ด้วย คำสั่งที่ส่งจากสแต็กการอ่านคำสั่งไบต์ที่ 1 จะเข้าสู่ตัวถอดรหัสคำสั่ง (Instruction Decoder) เพื่อทำการหาค่าตำแหน่งแอดเดรสต้นทางของคำสั่ง ตำแหน่งแอดเดรสปลายทางของหน่วยความจำที่ใช้เก็บผลการทำคำสั่ง และคำสั่งที่ใช้ในการทำงานของหน่วยประมวลผล

ในการรองรับการทำงานของแอดเดรสซึ่งโหมดต่างๆ อาศัยการถอดรหัสจากคำสั่งไบต์ที่ 2 โดยทำการถอดรหัสผ่านตัวถอดรหัสในโหมดต่างๆ โดยหากมีการเข้าถึงข้อมูลแบบไบต์ในตำแหน่งที่ต่ำกว่า 128 ซึ่งเป็นการเข้าถึงโดยตรงจะใช้การถอดรหัสโดย `direct_addr_dec` ซึ่งจะได้ผลเป็นตำแหน่งแอดเดรสในหน่วย ความจำส่งไปยังหน่วยความจำ

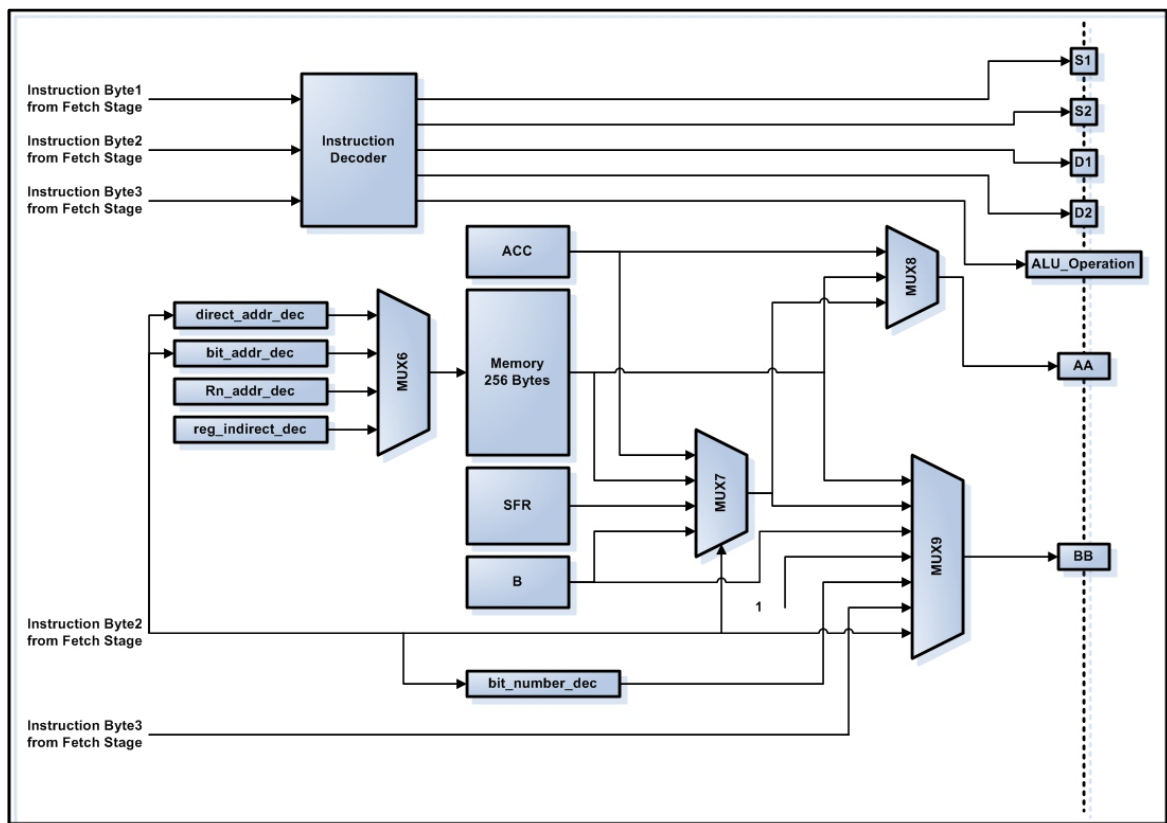
หากมีการอ้างถึงตำแหน่งของหน่วยความจำที่มีค่ามากกว่า 127 ซึ่งสามารถเข้าถึงได้โดยอ้อมเท่านั้นจะใช้ตัว `reg_indirect_dec` ในการถอดรหัสคำสั่งที่ทำงานในโหมดนี้ ปกติแล้วการทำงานของโหมดนี้จะต้องทำการอ่านข้อมูลในหน่วยความจำสองครั้ง ครั้งแรกเป็นการอ่านข้อมูลในรีจิสเตอร์ใช้งานทั่วไปซึ่งจะเก็บอยู่ใน R0 หรือ R1 เพื่อนำมาอ้างเป็นตำแหน่งของหน่วยความจำที่ใช้จริงอีกครั้ง ในการออกแบบได้ทำการเก็บหน่วยความจำ R0 และ R1 ไว้อีก 1 ชุดแล้วทำการส่งข้อมูลเข้ามาเพื่อให้สามารถหาตำแหน่งของข้อมูลที่ต้องการจริงส่งไปยังหน่วยความจำได้ทันที

สำหรับการเข้าถึงรีจิสเตอร์ใช้งานทั่วไป R0-R7 ซึ่งเป็นหน่วยความจำขนาด 8 บิตจำนวน 8 ตัวที่มีให้ใช้งานอยู่ 4 แบนกนั้น สามารถหาตำแหน่งได้โดยการใช้ `Rn_addr_dec` ซึ่งจะรับค่าของคำสั่งไบต์ที่ 1 ในตำแหน่ง 3 บิตสุดท้าย ในการบอกถึงรีจิสเตอร์ตัวใดตัวหนึ่งใน 8 ตัวและรับข้อมูลของแบนกที่ใช้งานปัจจุบันเข้ามาเพื่อให้สามารถหาตำแหน่งที่ต้องการได้ถูกต้อง

โครงสร้างของไมโครคอนโทรลเลอร์ต้นแบบได้อนุญาตให้มีการเข้าถึงข้อมูลในระดับบิตในตำแหน่งที่ 20H-2FH ได้โดยอาศัยคำสั่งแบบบิต จึงได้มีการกำหนดให้ `bit_addr_dec` ทำงานร่วมกับ `bit_number_dec` ในการถอดรหัสคำสั่งไบต์ที่ 2 ของคำสั่งแบบบิตไว้ โดยทำการถอดรหัสไบต์ของตำแหน่งในหน่วยความจำโดยใช้ `bit_addr_dec` เพื่อดึงข้อมูลในหน่วยความจำทั้งไบต์ออกมายังรีจิสเตอร์ AA และถอดรหัสตำแหน่งของบิตในไบต์ที่ต้องการด้วย `bit_number_dec` แล้วส่งไปเก็บไว้ในรีจิสเตอร์ BB

ในการเลือกว่าจะทำการเข้าถึงหน่วย ความจำในด้วยโหมดใดนั้นทำได้ด้วยการใช้ MUX6 เลือกตำแหน่งแอดเดรสที่ถอดรหัสจากตัวถอดรหัสโหมดต่างๆ ส่งไปยังหน่วยความจำจากนั้นก็จะได้รับค่าภายในหน่วยความจำกลับมาซึ่งสแต็กของการถอดรหัส ค่าที่ได้จะถูกจัดการด้วย MUX7 ถึง MUX9 เพื่อทำการเตรียมค่าโอเปอร์เรนด์ในสแต็กต่อไปโดย MUX8 และ MUX9 จะรับผิดชอบในการเลือกโอเปอร์เรนด์ตัวที่ 1 และ 2 ของคำสั่งส่งไปยังรีจิสเตอร์ AA และ BB ตามลำดับ

ค่าในรีจิสเตอร์ ในการทำงานของโหมด Immediate นั้น คำสั่งไบต์ที่ 2 หรือ 3 จะเป็นการเก็บค่าคงที่ที่ต้องการในการทำงานเอาไว้ ซึ่งจะถูกส่งไปเป็นโอเปอร์เรนด์ของสแต็กการทำคำสั่ง และเนื่องจากไม่มีกรณีใดๆ ที่ค่าของ Immediate จะเป็นโอเปอร์เรนด์ตัวแรกของการทำคำสั่ง ดังนั้นค่าคงที่ดังกล่าวจึงถูกจัดการด้วย MUX9 เพียงตัวเดียว เส้นทางหลักของข้อมูลในสแต็กการถอดรหัสคำสั่งแสดงดังรูปที่ 4.4

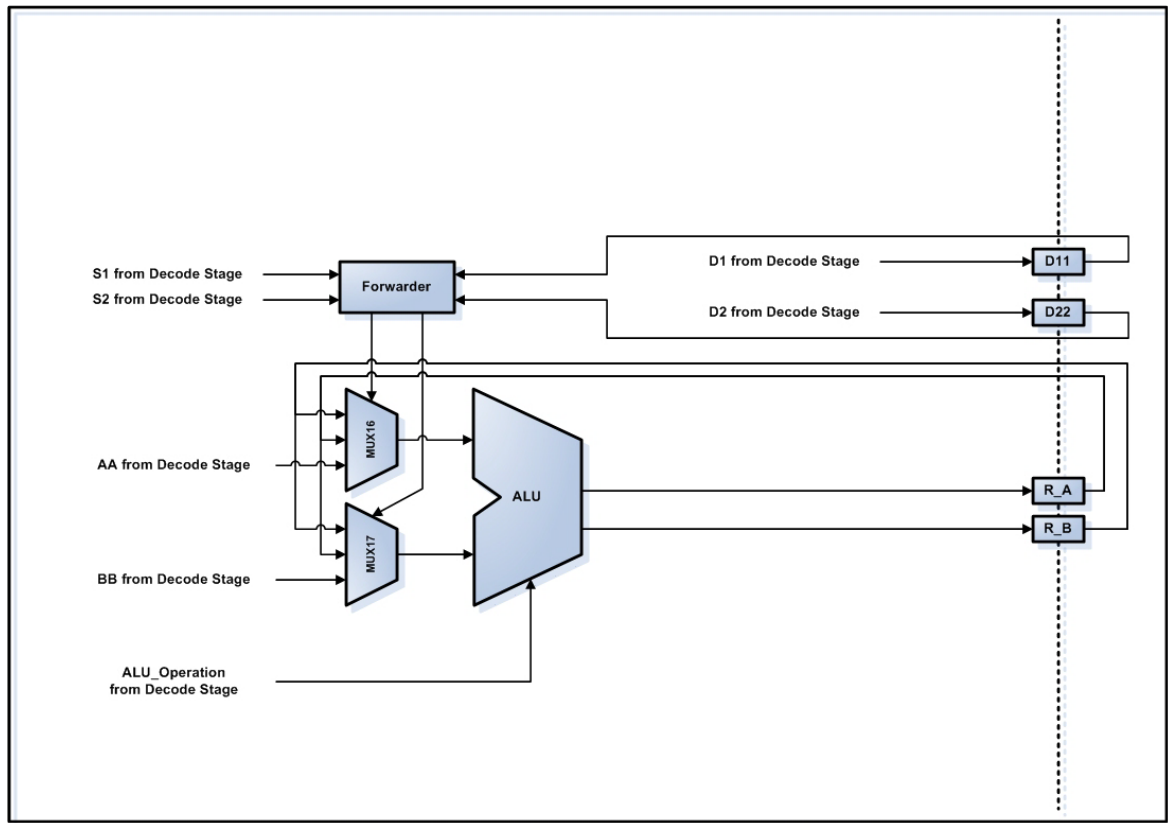


รูปที่ 4.4 เส้นทางหลักของข้อมูลในสแตจการอ่านคำสั่ง

4.2.3 สแตจการทำคำสั่ง (Execute Stage)

การทำงานหลักของสแตจการทำคำสั่งคือการประมวลผลทางคณิตศาสตร์และลอจิก ตามคำสั่งที่ส่งเข้ามาจากสแตจการถอดรหัสคำสั่ง เมื่อมีการทำคำสั่งประเภทเคลื่อนย้ายข้อมูล ALU ทำงานเป็นเพียงตัวส่งผ่านโอเพอร์แรนด์ไปยังเอาต์พุตเท่านั้น การทำคำสั่งแลกเปลี่ยนข้อมูล เช่น XCH ซึ่งมีปลายทางในการเขียนข้อมูลสองตำแหน่ง อาศัย ALU ซึ่งมีเอาต์พุตสองตัว ทำให้สามารถส่งข้อมูลไปเขียนยังตำแหน่งปลายทางทั้งสองได้ในเวลาเดียวกัน นอกจากนั้นแล้ว เอาต์พุตของ ALU ที่มีสองตัวยังเป็นประโยชน์ในการส่งข้อมูลที่ได้จากการทำคำสั่งเช่น DIV และ MUL ซึ่งมีการเก็บผลการคูณไบนารีและผลการหารไว้ในเอาต์พุตแรก และเก็บผลการคูณไบนารีและเศษจากการหารไว้ในเอาต์พุตที่สอง ALU ได้รับข้อมูลในการทำคำสั่งจากรีจิสเตอร์ AA และ BB ผ่านทาง MUX16 และ MUX17 โดยมี Forwarder ทำหน้าที่ในการตรวจสอบหากมีความต้องการที่จะฟอร์เวิร์ดข้อมูลโดยการเทียบจากค่าของรีจิสเตอร์ S1 และ S2 กับค่าของรีจิสเตอร์ D11 และ D22 โดยที่ค่าในรีจิสเตอร์ D11 และ D22 นั้นได้รับมาจาก D1 และ D2 ของสแตจการถอดรหัสคำสั่งที่ผ่านมา และส่งค่าต่อไปยังสแตจการเขียนผลลัพธ์เพื่อบันทึกผลการทำคำสั่งที่อยู่ใน R_A และ R_B ลงหน่วยความจำต่อไปส่วนของวงจรในการฟอร์เวิร์ดนั้นมีความสำคัญกับวงจรสแตจ

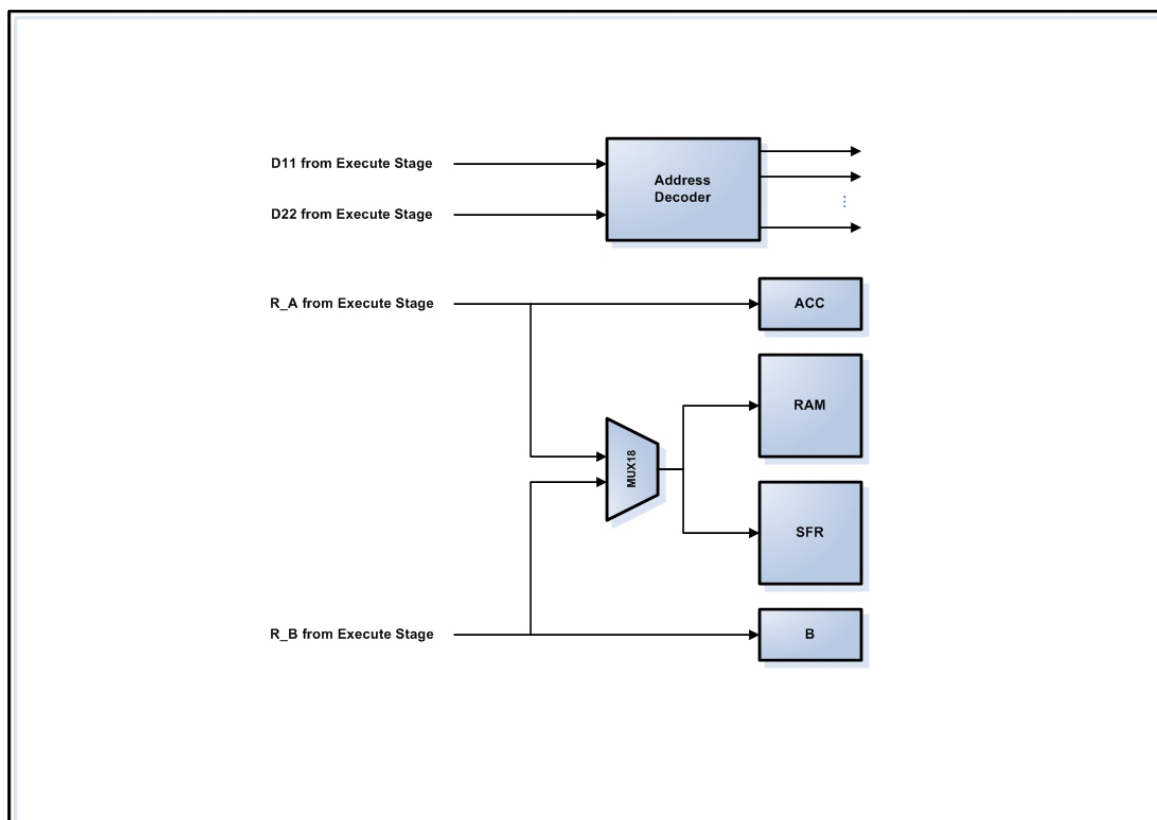
อีกด้วย ทั้งในสแตจการถอดรหัสคำสั่งและสแตจการอ่านคำสั่ง แต่ขอละไว้เพื่อความไม่ซับซ้อนในการอธิบายด้วยแผนผังวงจร เส้นทางหลักของข้อมูลในสแตจการทำคำสั่งแสดงดังรูปที่ 4.5



รูปที่ 4.5 เส้นทางหลักของข้อมูลในสแตจการทำคำสั่ง

4.2.4 สแตจการเขียนผลลัพธ์ (Writeback Stage)

การเขียนผลลัพธ์ของการทำคำสั่งต่างๆลงยังหน่วยความจำและรีจิสเตอร์จะถูกทำสแตจนี้ เมื่อมีการทำคำสั่งของซีพียูที่เกี่ยวข้องกับผลลัพธ์ทั้งสองตัวของ ALU ค่าของรีจิสเตอร์ R_A และ R_B จะถูกส่งไปเก็บยังรีจิสเตอร์ ACC และ B นอกจากนี้ หากเป็นการทำคำสั่ง XCH แล้วรีจิสเตอร์ ACC จะได้รับค่าของ R_A โดยตรงส่วนของหน่วยความจำภายในและรีจิสเตอร์ฟังก์ชันพิเศษนั้นจะ ได้รับข้อมูลหรือไม่ขึ้นกับคำสั่งที่งาน เส้นทางหลักของข้อมูลในสแตจการทำคำสั่งแสดงดังรูปที่ 4.6



รูปที่ 4.6 เส้นทางหลักของข้อมูลในสเตจการเขียนผลลัพธ์

4.3 ปัญหาและการแก้ปัญหาไปป์ไลน์

เมื่อออกแบบซีพียูให้มีการทำงานแบบไปป์ไลน์ย่อมมีโอกาสเกิดปัญหาของไปป์ไลน์ทั้ง 3 ด้านตามมา ในการออกแบบจึงกำหนดให้มีกลไกในการควบคุมและป้องกันการเกิดปัญหาดังกล่าวดังนี้

4.3.1 ด้านการควบคุม

ปัญหาด้านการควบคุมเกิดเมื่อมีการทำคำสั่งในกลุ่มกระโดด ซึ่งเป็นผลให้การทำงานของไปป์ไลน์ผิดไป จึงต้องมีการทิ้งผลของคำสั่งในสเตจต่างๆของไปป์ไลน์แล้วทำการอ่านคำสั่งที่ถูกต้องเข้ามาใหม่ แม้ว่าจะมีหลายวิธีการในการจัดการกับปัญหาด้านนี้ดังที่กล่าวไปแล้วในบทที่ 2 แต่ในการทำงานของกลไกดังกล่าวก็ยังไม่สามารถป้องกันปัญหาที่เกิดขึ้นได้สมบูรณ์ทั้งหมด คือยังมีโอกาสที่จะเกิดความผิดพลาดขึ้นได้ และหากเกิดขึ้นแล้วการแก้ไขก็มีขั้นตอนที่ยุ่งยากไม่น้อย

การแก้ปัญหาดังนั้นเพื่อเป็นการตัดปัญหาดังกล่าวในการออกแบบ จึงเลือกใช้การหยุดอ่านคำสั่งเข้าสู่ตัวจัดลำดับคำสั่ง และลดค่าของรีจิสเตอร์ที่ชี้กลับไปยังคำสั่งก่อนหน้านี้เพื่อรอให้ผลของการทำคำสั่งกระโดดเสร็จสิ้นลงก่อน และในระหว่างนั้นจะทำการป้อนคำสั่ง NOP ให้กับสเตจของไปป์ไลน์แทน การทำเช่นนี้ถือเป็นการยอม

เสียเวลาในการทำคำสั่ง NOP ที่เพิ่มขึ้นมาเอง ซึ่งลดประสิทธิภาพในการทำงานของซีพียูในด้านจำนวนคำสั่งที่ทำต่อจำนวนคำสั่งที่ให้ผลการทำงานจริง แต่หากมองในการทำงานแล้วการใช้ไปป์ไลน์นั้นช่วยให้ประสิทธิภาพการทำงานของซีพียูเพิ่มขึ้นมากอยู่แล้ว การเสียเวลากับคำสั่งกลุ่มนี้เพียงไม่กี่คำสั่งจึงถือว่ายอมรับได้ และเป็นผลดีกว่ายอมให้ปัญหานี้เกิดขึ้นแล้วตามแก้ไขในภายหลังซึ่งอาจเกิดปัญหาด้านอื่นตามมาอีกด้วย

4.3.2 ด้านการขึ้นต่อกันของข้อมูล

ปัญหาด้านการขึ้นต่อกันของข้อมูลเกิดเมื่อ มีการเขียนและอ่านข้อมูลกับหน่วยความจำซึ่งการออกแบบซีพียูตามโครงสร้างที่แบ่งไว้สามารถเกิดปัญหานี้ขึ้นได้เนื่องจากในจังหวะการทำงานใดจังหวะหนึ่ง จะมีการอ่านข้อมูลจากสเตจการถอดรหัสคำสั่งของคำสั่งที่ตามมา 2 คำสั่งหลังจากคำสั่งปัจจุบัน ในขณะที่เดียวกันก็มีการเขียนข้อมูลของสเตจการเขียนผลลัพธ์การทำงานคำสั่งปัจจุบันด้วย จึงได้ออกแบบการทำงานของซีพียูเพื่อป้องกันปัญหาด้านนี้ดังนี้

4.3.2.1 การแก้ปัญหา Read After Write

เลือกใช้การฟอร์เวิร์ดโอเพอร์เรนด์ เพื่อให้ข้อมูลที่ต้องการใช้งานของคำสั่งถัดไปซึ่งเป็นผลลัพธ์ของคำสั่งที่แล้ว แต่ยังไม่ได้ถูกเขียนสามารถส่งออกไปใช้ในการทำคำสั่งถัดไปได้ทันทีพร้อมๆ กับการเขียนผลลัพธ์ดังกล่าว โดยไม่ต้องรอการเขียนข้อมูลดังกล่าวลงหน่วยความจำแล้วค่อยทำการอ่านขึ้นมาทีหลังเป็นการป้องกันความผิดพลาดของข้อมูลเมื่อมีการเขียนช้ากว่าความต้องการใช้งาน และไม่จำเป็นต้องทำการหยุดการทำงานของไปป์ไลน์เพื่อรอข้อมูลดังกล่าวด้วย การทำฟอร์เวิร์ดโอเพอร์เรนด์อาศัยการทำงานของวงจรที่ออกแบบเพิ่มเติมอยู่นอกเหนือส่วนการทำงานของ สเตจปกติที่เรียกว่า Forwarder เช่น การทำงานของ Forwarder ภายในสเตจการทำคำสั่งจะทำการเปรียบเทียบค่าของตำแหน่งของข้อมูลที่จะต้องใช้หรือ Source Address ของคำสั่งที่จะทำกับ Destination Address ของคำสั่งที่ผ่านมา หากพบว่ามีความเกี่ยวข้องกัน จะทำการส่งค่าอยู่ที่ตำแหน่งดังกล่าวมาเป็นโอเพอร์เรนด์ในการทำงานของคำสั่ง แทนค่าที่ได้จากสเตจการถอดรหัสคำสั่งซึ่งเป็นค่าเก่าของข้อมูลที่เก็บอยู่ก่อนหน้านี้ ซึ่งการทำฟอร์เวิร์ดโอเพอร์เรนด์มีความสำคัญมาก เพราะเมื่อคำสั่งมีความสัมพันธ์เกี่ยวเนื่องกับข้อมูลเดิมๆ แล้ว หลีกเลี่ยงไม่ได้เลยที่จะต้องส่งข้อมูลที่ถูกต้องมาทำงานเพื่อให้ผลลัพธ์การทำงานถูกต้องด้วย

4.3.2.2 การแก้ปัญหา Write After Write

เนื่องจากซีพียูที่ออกแบบมีการทำงานตามลำดับของคำสั่งโดยไม่มีการข้ามขั้นตอนผลของการทำคำสั่งนั้นจึงออกมาเรียงตามลำดับด้วย ดังนั้นปัญหา Write After Write จึงไม่มีโอกาสเกิดกับซีพียูที่ออกแบบขึ้นมา

4.3.2.3 การแก้ปัญหา Write After Read

เป็นปัญหาในลักษณะที่ใกล้เคียงกับปัญหา Write After Write แต่เป็นปัญหาในการอ่านข้อมูลหลังการเขียนแทน และเนื่องจากซีพียูที่ออกแบบมีการทำงานตามลำดับของคำสั่ง ดังนั้นจึงไม่มีโอกาสเกิดปัญหานี้ขึ้น

4.3.3 ด้านโครงสร้าง

ปัญหาด้านโครงสร้างที่เกิดขึ้นกับซีพียูที่ทำงานแบบไปป์ไลน์ที่ออกแบบ นั้นเกี่ยวเนื่องจากการที่มีการเข้าถึงหน่วยความจำในหลายสเตจของการทำงาน ทั้งทำการอ่านข้อมูลในสเตจการถอดรหัสคำสั่งและการเขียน

ข้อมูลในสแต็กการเขียนผลลัพธ์ และการอ่านและเขียนหน่วยความจำโปรแกรมของสแต็กการอ่านคำสั่ง และการทำคำสั่งต่างๆ ต่างเกี่ยวข้องกับหน่วยความจำ ซึ่งนอกจากจะมีผลด้านการขึ้นต่อกันของข้อมูลแล้ว การใช้งานหน่วยความจำแบบปกติที่ใช้อยู่ทั่วไปนั้นจะไม่สามารถรองรับการทำงานตามลักษณะการออกแบบดังกล่าวนี้ ได้ทำให้เกิดปัญหาด้านโครงสร้างขึ้นได้เนื่องจากการทำงานที่ไม่สามารถรองรับการเข้าถึงได้จากหลายๆ ทาง

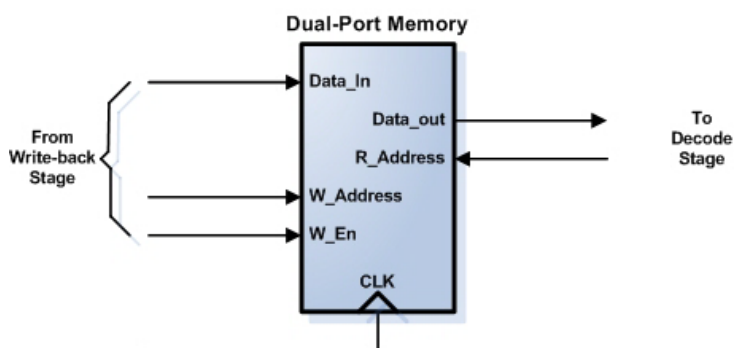
การแก้ปัญหาจึงทำการออกแบบหน่วยความจำให้เป็นแบบ Dual-port Memory คือสามารถทำการอ่านและเขียนข้อมูลได้พร้อมกัน เป็นลักษณะที่มีช่องทางการติดต่อกับหน่วยความจำสองทางคือเส้นทางการอ่านข้อมูลทางหนึ่ง และการเขียนข้อมูลอีกทางหนึ่ง แต่การอ่านและเขียนข้อมูลนั้นกำหนดให้มีการเขียนข้อมูลก่อน ในช่วงของสัญญาณเป็น High ส่วนการอ่านสามารถทำได้ทั้งสองช่วงสัญญาณ เพื่อช่วยป้องกันปัญหาด้านการขึ้นต่อกันของข้อมูลด้วย

4.4 รายละเอียดของการออกแบบเพิ่มเติม

4.4.1 การออกแบบหน่วยความจำ

การออกแบบหน่วยความจำ มีการใช้หน่วย แบบ Dual-port ดังรูปที่ 4.7 ซึ่งสามารถทำการอ่านและเขียนได้พร้อมกันในหนึ่งช่วงของสัญญาณ ซึ่งการเข้าถึงข้อมูลใน หน่วยความจำนั้น การเขียนจะได้สิทธิ์ในการเข้าถึงข้อมูลก่อนการอ่าน นอกจากนี้มีสร้าง Register ของตัวแปรที่มีการใช้บ่อยครั้ง ออกจากหน่วยความจำโดยตรง เช่น รีจิสเตอร์ A, รีจิสเตอร์ B, PSW เป็นต้น ซึ่งจะมีการควบคุมการอ่านจากส่วนของ Decode Stage และเขียนส่วนของ Writeback Stage จากคอนโทรลยูนิต

การเข้าถึงหน่วยความจำหลักทำโดยสแต็กการถอดรหัสและสแต็กการเขียนผลลัพธ์ในการทำงานปกติ ส่วนการอ่านค่าของรีจิสเตอร์ใช้งานทั่วนั้นสามารถเรียกใช้งานได้โดยตรง เช่น ใช้เพื่อนำมาใช้ในการทำคำสั่งแบบเรียกแอดเดรสโดยอ้อม นอกจากนี้ภายในตัวของ mem_unit เอง จะมีอ้างการเข้าถึงตำแหน่งข้อมูลได้ถึง 2 ตำแหน่งพร้อมกัน ซึ่งมีประโยชน์อย่างมากต่อการย้ายค่า ตัวแปร ประเภท 16 บิต เข้า-ออก หน่วย ความจำ แบบ 8 บิต ได้ภายในครั้งเดียว



รูปที่ 4.7 หน่วยความจำแบบ Dual-Port

4.4.2 การออกแบบคอนโทรลยูนิต

การออกแบบคอนโทรลยูนิตที่นิยมใช้กันนั้นมี 2 วิธีคือ การใช้ Hard-wired or Hardware Control Unit และการใช้ Firmware Micro Programmed or Microcode Control Unit ในการออกแบบนี้จะเลือกใช้การออกแบบคอนโทรลยูนิตแบบ Hard-wired เนื่องจากให้ความเร็วในการทำงานที่สูง แม้ว่าจะมีความยุ่งยากซับซ้อนในการออกแบบมากกว่าแบบ Firmware Micro Programmed แต่อาศัยการเขียนโปรแกรมที่มีลักษณะของการเรียกใช้โปรแกรมที่ช่วยในการลดความซับซ้อนได้ระดับหนึ่ง การทำงานภายในของคอนโทรลยูนิตนั้นจะใช้ในการควบคุมการทำงานต่างๆ โดยแบ่งตามแอดเดรสโหมดและสเตจการทำงานหลักๆ ซึ่งได้รวมการทำงานของคอนโทรลยูนิตปกติ และการควบคุมปัญหาไปป์ไลน์ (Hazard Control) เอาไว้ด้วยกัน